

1 Search and Arc Consistency

Recall: we're dealing with CSPs, which consist of

- a set of variables;
- a domain of possible values for each variable; and
- a set of constraints that an acceptable solution must meet.

Let's denote these as a triple (X, D, C) where X is a set of variables, D is a set of domains for the variables, and C is a set of constraints.

1.1 Search revisited

We talked a bit about using search to solve CSPs. Let's review a bit, and then expand more formally on **backtracking search**, which we saw the seeds of before as *combinatorial search*.

1.1.1 Reminder: generate-and-test

Intuition: we generate every possible solution within the joint domains of the variables and test to see if each is a solution.

For simplicity today we're going to use a small arithmetic example CSP:

Variables: $X = \{a, b, c\}$;

Domains: $D(a) = D(b) = D(c) = \{1, 2, 3\}$;

Constraints: $C = \{C_1, C_2\}$, where $C_1 : a < b$ and $C_2 : b < c$.

How might we generate all solutions? If we want to hard-code for this problem, then something like:

```
for value_a in D(a):
    for value_b in D(b):
        for value_c in D(c):
            if value_a < value_b and value_b < value_c:
                return value_a, value_b, value_c
return UNSAT
```

Not great. In the worst case this will be a large number of checks, and is inelegant to implement. For a larger problem there will be far too many checks to be feasible.

1.1.2 Revisit: backtrack search

We saw a better (but still asymptotically exponential time) search in week 1 as ‘combinatorial search’ – here we’ll look at this a little more formally and include a little more pruning, and call it **backtrack search**.

The idea here is to search the space via depth-first search, but give up on a branch (and ‘prune’ it and ‘backtrack’) when we know it can’t be part of any solution.

- One time when we know it can’t be part of any solution: when we have fixed all the variables involved in some constraint, and that constraint isn’t satisfied.

Example on our example CSP: a partial solution that has $a = 2$ and $b = 1$ can’t be part of any solution and can be pruned.

We will work through a hand-drawn example in lecture.

Why do we need to do anything else?

- this search will often be too slow.

There are many approaches to help us go faster!

- successor ordering
- arc consistency ←
- fancier consistencies

1.2 Arc consistency (AC)

The idea of arc consistency is to decrease the size of the domains before we start searching – sometimes we even get lucky and solve the problem without any search.

1.2.1 Constraint networks

We can represent the relationships between variables and constraints as a (bipartite) graph, and then we will examine each edge (or *arc*) to compute arc consistency.

In our **constraint network**:

- each variable is a node (conventionally a circle);
- each constraint is a node (conventionally a rectangle);
- there is an edge between a variable node and a constraint node when that variable is involved in that constraint.

We will work through a hand-drawn example in lecture.

Then we say that an arc between a variable x and a binary constraint $C_i(x, y)$ is **arc consistent** if for each value in the domain of x there is a value in the domain of y that satisfies C_i .

We'll look at our example for arcs where that might not be true for our full domains.

We say our whole CSP is arc consistent if every arc is.

So what do we do if our CSP is not arc consistent?

- we can make our CSP arc consistent!
- remove every value from a domain that is not consistent.
 - but then we might need to adjust other domains, as we might have made other arcs inconsistent.

Which arcs do we need to reconsider? Not all of them!

- if we remove a value from the domain of variable x , we need only reconsider arcs from other variables to constraints that also involve x .

To think about: why do we not yet need to consider other arcs from x to other constraints?

We will work through a hand-drawn example in lecture.

1.2.2 Algorithm for AC

We can now write out an algorithm for enforcing arc consistency:

```
function ac(X, D, C):
    # Takes: variables X, domains D, constraints C
    # Gives: arc consistent domains for variables
    set D_new gets empty set
    set todo gets empty set
    for each variable x in X:
        D_new[x] gets D[x]
        todo gets all arcs from x to a constraint involving x
    while todo is not empty:
        get an arc (x, c) from todo
        remove (x, c) from todo
        y is the other variable involved in c
        domain_changed gets False
        for every value_x in D_new[x]:
            remove_me gets True
            for every value_y in D_new[y]:
```

```

        if value_x, value_y satisfy c:
            remove_me gets False
            stop loop
    if remove_me:
        remove value_x from D_new[x]
        domain_changed gets True
if domain_changed:
    add all arcs from other variables to
    constraints involving x to todo

```

This procedure is essentially the classic **AC-3** algorithm (Mackworth, 1977).

There are three possible outcomes after we run this algorithm:

1. every domain has only one value;
2. at least one domain has no values;
3. all domains have at least one value, some have more.

What do these mean?

A few time complexity questions to ponder as we look at examples:

- why will this halt?
- how long will this take?

1.2.3 Examples

For the examples below, write $\text{revise}(x, y)$ for the step “remove any value from $D(x)$ that has no supporting value in $D(y)$ ”, i.e. processing the arc from x to the constraint it shares with y .

Example 1: a formal working of our on-going simple integer example. $X = \{a, b, c\}$, $D(a) = D(b) = D(c) = \{1, 2, 3\}$, $C_1 : a < b$, $C_2 : b < c$.

- $\text{revise}(a, b)$: $a = 3$ has no $b > 3$ available, so remove it. $D(a) = \{1, 2\}$. $D(a)$ changed, so re-queue $\text{revise}(b, a)$.
- $\text{revise}(b, a)$: $b = 1$ has no $a < 1$ available, so remove it. $D(b) = \{2, 3\}$. $D(b)$ changed, so re-queue $\text{revise}(a, b)$ and $\text{revise}(c, b)$.
- $\text{revise}(b, c)$: $b = 3$ has no $c > 3$ available, so remove it. $D(b) = \{2\}$. Re-queue $\text{revise}(a, b)$ and $\text{revise}(c, b)$.
- $\text{revise}(c, b)$: with $D(b) = \{2\}$, $c = 1$ and $c = 2$ have no supporting b (need $b < c$), so remove both. $D(c) = \{3\}$. Re-queue $\text{revise}(b, c)$.

- $\text{revise}(a, b)$: with $D(b) = \{2\}$, $a = 2$ has no $b > 2$ available, so remove it. $D(a) = \{1\}$. Re-queue $\text{revise}(b, a)$.
- $\text{revise}(b, c)$, $\text{revise}(b, a)$: no further removals.

Final domains: $D(a) = \{1\}$, $D(b) = \{2\}$, $D(c) = \{3\}$ – outcome (1). Arc consistency alone found the unique solution $a = 1, b = 2, c = 3$, with **no search required**.

Example 2: another simple integer example, this time showing arc consistency isn't always enough. Take a triangle of “not equal” constraints: $X = \{a, b, c\}$, $D(a) = D(b) = D(c) = \{1, 2\}$, $C = \{a \neq b, b \neq c, a \neq c\}$.

- For every arc $\text{revise}(x, y)$: each value in $D(x)$ always has a supporting, different value in $D(y)$, because both domains are $\{1, 2\}$. Nothing is ever removed.

Final domains: $D(a) = D(b) = D(c) = \{1, 2\}$ – outcome (3), and the CSP is already arc consistent. But there is in fact **no solution**: this is a triangle (an odd cycle), and three mutually-adjacent variables can't take pairwise-different values from a domain of size 2. Arc consistency only checks pairs of variables, so it can't see this – backtracking search would still need to try, e.g., $a = 1, b = 2$, then discover c has nothing left to be assigned ($c \neq a$ and $c \neq b$ rules out both remaining values) and backtrack. This is why outcome (3) doesn't mean “solvable”, only “not yet disproved by arc consistency”.

Example 3: a graph-colouring example. Take a path of three vertices, $u_1 - u_2 - u_3$, where u_1 has already been fixed to colour 1 (perhaps by an earlier decision). $D(u_1) = \{1\}$, $D(u_2) = \{1, 2\}$, $D(u_3) = \{1, 2, 3\}$, constraints $u_1 \neq u_2$ and $u_2 \neq u_3$.

- $\text{revise}(u_2, u_1)$: $u_2 = 1$ has no supporting $u_1 \neq 1$ (since $D(u_1) = \{1\}$), so remove it. $D(u_2) = \{2\}$. Re-queue $\text{revise}(u_3, u_2)$.
- $\text{revise}(u_1, u_2)$: no removals ($u_1 = 1$ is still supported by $u_2 = 2$).
- $\text{revise}(u_3, u_2)$: with $D(u_2) = \{2\}$, $u_3 = 2$ has no supporting $u_2 \neq 2$, so remove it. $D(u_3) = \{1, 3\}$. Re-queue $\text{revise}(u_2, u_3)$.
- $\text{revise}(u_2, u_3)$: no further removals.

Final domains: $D(u_1) = \{1\}$, $D(u_2) = \{2\}$, $D(u_3) = \{1, 3\}$ – outcome (3). Notice how fixing u_1 's colour propagated one step down the path and shrank u_2 's domain, which in turn shrank u_3 's domain: this cascading is exactly why arc consistency can prune far more than checking each constraint once.

1.2.4 Time complexity

So why will AC halt?

- in short, because we cannot remove things from domains indefinitely. Informally:
 - if we ever do an iteration with no removals and todo is empty, then we stop;
 - every time we remove something a domain gets smaller: eventually we end up with empty domains.

Or, thought about another way: every iteration either removes something from a domain or doesn't add anything to todo. Therefore there are a finite bounded number of times we add things to todo, because the sum of the domain sizes is finite.

Then what is the worst-case time complexity, on a problem with n variables, d maximum domain size, and c number of constraints?

- when we check our domains for consistency, this takes $O(d^2)$;
- how many things will we add to our todo queue? Consider it from a degree perspective: $O(cd)$;
- so $O(cd^3)$.